

PROJECT PROPOSAL

AI Workforce OS

An Enterprise Multi-Agent Platform for Business Operations

Project Title: AI Workforce OS: An Enterprise Multi-Agent Platform for Business Operations

Course: Web Services & Service-Oriented Architecture (IT644), Autumn 2026

Team Size: 8 Members

Team Members

Name	Student ID
Aum Parmar	202512004
Param Patel	202512005
Panthil Shah	202512050
Love Shah	202512058
Afif Momin	202512063
Ismail Mansuri	202512075
Yash Doshi	202512094
Fahim Saiyad	202512103

Table of Contents

1. Short Description	3
2. Application Type	3
3. Motivation / Problem Statement	3
4. Target Users	4
5. Use Cases	4
6. Scope & High-Level Features	5
7. Technology Stack	6
8. Roles & Responsibilities	7
9. Timeline & Milestones	7

1. Short Description

AI Workforce OS is a web platform on which a company sets up a team of specialized AI agents, referred to as AI employees, and assigns them day-to-day business work. The platform provides agents for common roles such as HR, engineering management, software development, research, marketing, and customer support. Agents share a common memory so that context carries over from one task to the next, they answer questions from the company's own documents using retrieval-augmented generation (RAG), and they carry out actions in real business tools (email, calendars, Jira, GitHub, Slack, social media, CRM) through Model Context Protocol (MCP) integrations. Every action is permission-checked and logged, and sensitive actions wait for human approval before they run.

The backend follows a service-oriented architecture. Authentication, organization management, agent orchestration, knowledge retrieval, shared memory, tool integration, and analytics are separate services that communicate over documented REST APIs and Kafka events. Each service can be developed, tested, deployed, and scaled independently, which also allows the eight of us to work in parallel against agreed API contracts.

2. Application Type

The system is delivered as a responsive web platform with an API-first backend. This matches the course preference of a web application for the administrative side and keeps the semester scope realistic.

- **Responsive web application.** A single Next.js application serves both the admin console (agent configuration, permissions, approvals, audit logs) and the employee workspace (chat with agents, tasks, dashboards). The layout is mobile-responsive, so use on a phone browser is covered without a separate native app; a dedicated mobile app is left as future work.
- **REST API services.** The backend is a set of independently deployable FastAPI services, each documented with Swagger/OpenAPI, so the web client and any future client consume the same contracts.
- **Event-driven backbone.** Long-running, multi-agent workflows communicate through Kafka topics, which decouples the services that produce work from the agents that consume it.
- **Cloud-native deployment.** Each service is containerized with Docker and deployed on Kubernetes, so heavy services such as agent inference and RAG retrieval can scale without over-provisioning the lighter ones.

3. Motivation / Problem Statement

Most businesses run on a patchwork of disconnected tools: email, chat, ticketing, code hosting, CRM, social media, and spreadsheets. Moving information between these tools falls on people, and much of that work is repetitive. Existing AI assistants help only at the margins, because each one is a single chatbot: it cannot coordinate with other assistants, it does not remember organizational context, and giving it direct access to business systems raises obvious safety concerns. In particular:

- Information and actions are spread across many applications, and there is no single place where a piece of work can be handed over from start to finish.
- Routine knowledge work such as candidate screening, sprint bookkeeping, market research, social media posting, ticket triage, and KPI reporting consumes hours of skilled staff time every week.
- Company knowledge sits scattered across Drive, Notion, Confluence, and wikis, so the quality of an answer depends on who is asked.
- Current assistants have no shared memory and no agent-to-agent handoff, so a human still relays context between the steps of any multi-role workflow.

- Automation without scoped permissions, approval gates, and audit trails is an operational and security risk that most organizations rightly refuse to accept.
- There is no single dashboard that shows what the automation did, on whose authority, and with what result.

AI Workforce OS addresses these problems with a team of role-specialized agents that collaborate through shared memory, answer from a common knowledge base, and act on business tools only through scoped MCP permissions, with human approval for sensitive actions and a complete audit log of everything that ran.

4. Target Users

User Group	Description	How They Benefit
Startups and SMEs (founders, owners)	Small organizations that cannot yet justify dedicated HR, marketing, research, or support staff.	Routine functions are delegated to AI employees, and one dashboard shows the state of all of them.
Department managers	Operations, engineering, marketing, and support leads inside larger teams.	Repetitive coordination (sprint bookkeeping, reporting, triage, campaign operations) moves to agents; the manager reviews and approves instead of producing.
Employees and knowledge workers	Human staff who work alongside the AI workforce every day.	They ask questions grounded in company documents and hand routine tasks to agents.
Administrators and IT / security teams	Staff responsible for governance of the platform and its tool access.	Role-based access control, scoped MCP permissions, approval workflows, and complete audit logs.

5. Use Cases

The table below lists the core use cases, ordered roughly by the lifecycle of an organization on the platform.

#	Use Case	Primary Actor	Description
1	Organization onboarding	Org Admin	Create a company workspace, invite members, and assign roles.
2	Login and access control	User	Authenticate with JWT/OAuth2; role-based access applies to every view and API.
3	Configure an AI employee	Org Admin	Enable an agent and set its goals, model, tool permissions, and approval policy.
4	Connect knowledge sources	Org Admin	Link Drive, Notion, Confluence, GitHub wikis, and documents for ingestion into the knowledge base.
5	Ask enterprise questions	Employee	Receive answers grounded in company documents, with citations.
6	Delegate a task to an agent	Manager	Describe a task in natural language; the agent plans it and carries it out.
7	Run a multi-agent workflow	Agent Orchestrator	Split one goal into subtasks handled by several agents: research, campaign draft, review, publication.
8	Approve agent actions	Manager / CEO Agent	Review outbound or high-impact actions in an approval queue before they execute.

#	Use Case	Primary Actor	Description
9	HR operations	HR Agent	Screen applications, draft onboarding emails, and schedule interviews through the Gmail and Calendar integrations.
10	Sprint and ticket management	Engineering Manager Agent	Create and update Jira tickets, review GitHub pull requests, and post standup summaries to Slack.
11	Market and technical research	Research Agent	Compile competitor and market reports from web research into shareable documents.
12	Campaign publishing	Marketing Agent	Draft posts and, after approval, publish them to Instagram and LinkedIn.
13	Support ticket resolution	Customer Support Agent	Triage tickets, draft replies from the knowledge base, and escalate edge cases.
14	KPI dashboards and reports	Analytics Agent	Aggregate agent and business activity into dashboards and periodic reports.
15	Audit log review	Org Admin	Inspect every agent decision, tool call, and approval with full traceability.

6. Scope & High-Level Features

Features are organized into modules that map onto the platform's services. The scope is phased so that one semester stays realistic: the core scope is committed for the mid-semester demonstration and the final delivery, and the extended scope is taken up as time permits.

Organization and User Management

- Company workspace creation, member invitations, and profile management
- Role assignment (admin, manager, employee) with per-agent permission grants
- Organization-level settings: approval policies, working hours, integration credentials

Authentication and Security

- JWT-based sessions with OAuth2 login, password reset, and multi-device handling
- Role-based access control enforced at the API gateway and inside every service
- Scoped MCP permissions per agent, secret management, encrypted memory, and audit logs

Agent Orchestration and Shared Memory (core)

- An orchestrator service that routes goals, breaks tasks down, and coordinates handoffs between agents
- A shared memory layer on PostgreSQL and Redis, so agents retain organizational and task context
- Kafka-backed asynchronous workflows for long-running, multi-step jobs

AI Employees

- **Core set:** HR, Engineering Manager, Developer, Research, Marketing, and Customer Support agents
- **Extended set:** CEO, COO, Product Manager, Sales, Finance, Legal, DevOps, and Analytics agents
- Per-agent persona, goals, tool permissions, and approval policy

Enterprise Knowledge Base (core)

- An ingestion pipeline covering document loading, chunking, embedding, and storage in the Qdrant vector database, with retrieval that cites its sources
- Connectors for Google Drive, Notion, Confluence, GitHub wikis, PDF, and DOCX
- Knowledge-graph enrichment in Neo4j for entity and relationship queries (extended)

MCP Tool Integrations (core subset first)

- Gmail (read, draft, send), Google Calendar (scheduling), Slack (read, send)
- GitHub (issues, pull requests, repositories), Jira (create, update), Google Drive (read, upload)
- Instagram and LinkedIn publishing with analytics; CRM and AWS/Kubernetes operations (extended)

Approvals, Governance and Observability

- An approval queue where humans review outbound and high-impact actions before they run
- A complete audit trail of agent reasoning steps, tool calls, and outcomes
- Platform health and usage monitoring with Prometheus and Grafana

Admin Dashboard and Analytics

- Agent configuration, permission, and approval management views
- KPI dashboards: tasks completed, hours saved, per-agent activity, integration usage
- Organization-wide reporting compiled by the Analytics agent (extended)

Out of scope for this semester: voice agents, meeting automation, a native mobile application, multi-tenant billing, an agent marketplace, and a third-party plugin SDK.

7. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js (React, TypeScript), Tailwind CSS	Responsive web application for the admin console and employee workspace
Backend	FastAPI (Python)	Independent REST API services
Architecture	Service-Oriented Architecture (SOA)	Loosely coupled services with documented contracts
Agent framework	LangGraph / OpenAI Agents SDK	Agent workflows, tool use, and multi-agent orchestration
LLM providers	OpenAI / Anthropic Claude APIs	Language models that power the agents
APIs	REST, Swagger / OpenAPI	Inter-service and client-service communication and documentation
Authentication	JWT, OAuth2, RBAC	Sessions, third-party login, role-gated access
Databases	PostgreSQL, Redis	Relational data; caching and short-term agent memory
Vector and graph stores	Qdrant, Neo4j	Embedding search for RAG; knowledge-graph queries
Messaging	Apache Kafka	Event-driven communication for multi-agent workflows
Integrations	MCP servers: Gmail, Calendar, Slack, GitHub, Jira, Drive, Instagram, LinkedIn, CRM, AWS	Scoped agent access to business tools

Layer	Technology	Purpose
DevOps	Docker, Kubernetes, GitHub Actions	Containerization, orchestration, CI/CD
Cloud	AWS, using free tiers and student credits where possible	Hosting with a public URL
Observability	Prometheus, Grafana, OpenTelemetry	Metrics, dashboards, and distributed tracing
Testing	Pytest, Postman	Unit, integration, and API testing
Design and PM	Figma, Jira, Git/GitHub	UI/UX prototyping, sprint tracking, version control

8. Roles & Responsibilities

The team is organized around the platform's service boundaries, so members can work independently against shared API contracts. The rows are ordered so that each adjacent pair forms a natural working unit: architecture with orchestration, the agents with the knowledge base they draw on, the tool integrations with the frontend that surfaces them, and the backend platform with the operations that deploy it.

Team Member	Student ID	Role	Key Responsibilities
Aum Parmar	202512004	Project Lead and System Architecture	Overall architecture, service boundaries and API contracts, sprint planning, integration oversight, LangGraph workflows
Love Shah	202512058	Agent Orchestration and Shared Memory	Orchestrator service, agent-to-agent communication, PostgreSQL/Redis memory layer, web integration
Panthil Shah	202512050	AI Employee Agents	Personas, prompts, and tool logic for the core agents; agent evaluation, quality, and documentation
Fahim Saiyad	202512103	Knowledge Base and RAG Pipeline	Ingestion connectors, chunking and embeddings, Qdrant retrieval, citations, Neo4j enrichment
Ismail Mansuri	202512075	MCP Integration Layer	MCP servers (Gmail, Calendar, Slack, GitHub, Jira, Drive, social platforms), permission scopes, approval hooks
Yash Doshi	202512094	Frontend (Next.js Dashboard)	Admin console and employee workspace UI, approvals queue, dashboards, responsive design
Param Patel	202512005	Backend Platform: Gateway, Auth and Organizations	API gateway, authentication service (JWT/OAuth2), organization and user services, RBAC
Afif Momin	202512063	DevOps, Testing and Documentation	Docker/Kubernetes deployment, CI/CD, Kafka and monitoring setup, test plans, API documentation

In addition, every member writes unit tests and documentation for their own services and contributes to the weekly progress reports, the mid-semester demonstration, and the final report and presentation.

9. Timeline & Milestones

The timeline is anchored to the course deliverable deadlines. Development milestones are planned backward from the mid-semester demonstration and the final delivery.

Week(s)	Milestone	Key Deliverables
1–2 (27 Jul–9 Aug)	Proposal and Project Plan	Project proposal PDF (due 5 Aug); Git repository and Jira board setup; task timeline and assignments (due 9 Aug)
3 (10–16 Aug)	Requirements Specification	BRD with functional and non-functional requirements, user stories, use-case diagrams, acceptance criteria, initial wireframes (due 16 Aug)
4–5 (17–30 Aug)	System Design	Architecture diagram, ER schema, OpenAPI contracts, component diagrams, near-final Figma mockups (due 27 Aug)
6–8 (31 Aug–20 Sep)	Platform Core	API gateway, authentication and RBAC, organization service, orchestrator skeleton, shared memory layer, CI pipeline
9–10 (21 Sep–4 Oct)	Knowledge Base and First Agents	RAG ingestion pipeline with Qdrant retrieval and citations; the Research and HR agents working end to end
11–12 (5–18 Oct)	MCP Integrations and Mid-Semester Demo	Core MCP servers, approval workflow, remaining core agents; mid-semester demonstration (week of 12 Oct)
13–14 (19 Oct–1 Nov)	Extended Capabilities	Extended agents, Kafka multi-agent workflows, analytics dashboards, notifications
15 (2–8 Nov)	Testing	Test plan, unit and integration test suites, seeded database, UI testing, bug log (due week of 2 Nov)
16 (9–15 Nov)	Deployment and Hardening	Kubernetes deployment on AWS with a public URL, monitoring, security review, user manual, API documentation
17 (16–19 Nov)	Final Delivery	Final application and hosting (16 Nov); final report (18 Nov); presentation, demo and poster (18 Nov onwards); peer reviews (19 Nov)

Weekly one-page progress reports are submitted throughout the project (27 Jul to 15 Nov) as required by the course deliverables document. In line with the course's preference for AI-assisted work, the team uses AI tools (Anthropic Claude) for design, documentation, coding, and testing, and submits its prompt log alongside every deliverable.